

DOCUMENTAÇÃO TÉCNICA

Axel CRM

Plataforma multi-tenant de gestão comercial, operacional e financeira — Angular 18, Spring Boot 4 e PostgreSQL.

Escopo: arquitetura, segurança e multi-tenancy, modelo de dados, API REST, frontend, regras de negócio, ambiente e deploy, guia de desenvolvimento e pontos de atenção.

Base: comportamento verificado no código-fonte do repositório `axel-crm`, branch `master`.

Gerado em: 07 de agosto de 2026

Sumário

Axel CRM — Documentação Completa do Sistema

Índice

- 1. Visão Geral
 - 2. Arquitetura
 - 3. Segurança e Multi-Tenancy
 - 4. Modelo de Dados
 - 5. API REST
 - 6. Frontend
 - 7. Regras de Negócio
 - 8. Ambiente e Deploy
 - 9. Guia de Desenvolvimento
 - 10. Pontos de Atenção
 - 11. Onboarding Rápido
 - 12. Anexos
-

Axel CRM — Documentação Completa do Sistema

Versão: 2.0 — consolidada em 25/08/2026
Stack: Angular 18 SPA + Spring Boot 4 REST API + PostgreSQL 16+ Flyway
Idioma: pt-BR (interface, mensagens de negócio e moeda BRL)
Repositório: `axel-crm/` com `backend/`, `frontend/`, `docs/`, `render.yaml`

Este documento consolida **toda** a documentação técnica do Axel CRM em um único arquivo navegável. Cada seção corresponde a um capítulo da pasta `docs/` e foi verificada contra o código em `backend/` e `frontend/`. Para o manual voltado ao usuário final, veja `manual-do-usuario/README.md`.

Índice

- 1. Visão Geral
- 2. Arquitetura
- 3. Segurança e Multi-Tenancy
- 4. Modelo de Dados
- 5. API REST
- 6. Frontend
- 7. Regras de Negócio
- 8. Ambiente e Deploy
- 9. Guia de Desenvolvimento
- 10. Pontos de Atenção
- 11. Onboarding Rápido
- 12. Anexos: ADRs, WhatsApp e Geração de PDF

1. Visão Geral

1.1 O que é o Axel CRM

Plataforma **multi-tenant** que reúne num único workspace autenticado três domínios que normalmente ficam em sistemas separados:

Domínio	Módulos
Comercial	Prospecção, parceiros/indicadores, clientes, contatos, leads, negócios e pipelines

Domínio	Módulos
Operacional	Projetos, produtos, contratos, processos jurídicos, tarefas, propostas, campanhas, tickets, documentos, agenda e WhatsApp Inbox
Financeiro	Faturamento (invoices), transações, plano de contas, contas bancárias, apontamento de horas, comissões e relatórios (DRE/DFC, fluxo de caixa)

O ciclo ponta-a-ponta suportado é:

Prospect → Lead → Negócio (Deal) → Proposta → Contrato → Projeto → Fatura → Relatório
 ↳ Cliente (conversão) ↳ PDF público

1.2 Usuários e superfícies

Primário: operadores internos (vendas, operações, financeiro) compartilhando a mesma organização, uso desktop-first diário.

Secundárias:

Superfície	Rota Frontend	Autenticação
Portal do cliente	<code>/portal/client</code>	Sessão do portal
Portal do parceiro	<code>/portal/partner</code>	Sessão do portal
Proposta pública	<code>/public/proposals/:token</code>	Token UUID na URL, sem login

1.3 Papéis

Enum `Role` em `commons/entity/Organization.java` e backend `entity/enums/Role.java`:

`SUPER_ADMIN`, `ADMIN`, `MANAGER`, `SALES`, `SUPPORT`, `USER`, `VIEWER`

Controle efetivo hoje: `adminGuard` no frontend restringe `/users` e `/integrations` a `ADMIN` / `SUPER_ADMIN`. API autentica mas **não autoriza por papel** — ver [Pontos de Atenção](#).

1.4 Stack técnico

Camada	Tecnologia	Versão/Detalhe
Frontend	Angular (standalone) + Angular Material + Chart.js + RxJS	18.2 / 18 / 4 / 7
Backend	Spring Boot + Spring Security + Spring Data JPA + Lombok	4.0.7 + Java 21
Banco	PostgreSQL + Flyway	16+ / V1–V29 + R_seed
Auth	JWT HMAC256 (Auth0 <code>java-jwt</code>) + BCrypt	4.4.0
PDF	OpenPDF (propostas e faturas)	—

Camada	Tecnologia	Versão/Detalhe
Docs API	SpringDoc OpenAPI / Swagger UI	2.8.0
Deploy	Render Blueprint (<code>render.yaml</code>) + Docker multi-stage	—
Testes	JUnit 5 + Mockito + H2 (backend), Karma+Jasmine (frontend)	144 testes / 23 classes

1.5 Números do sistema (verificados)

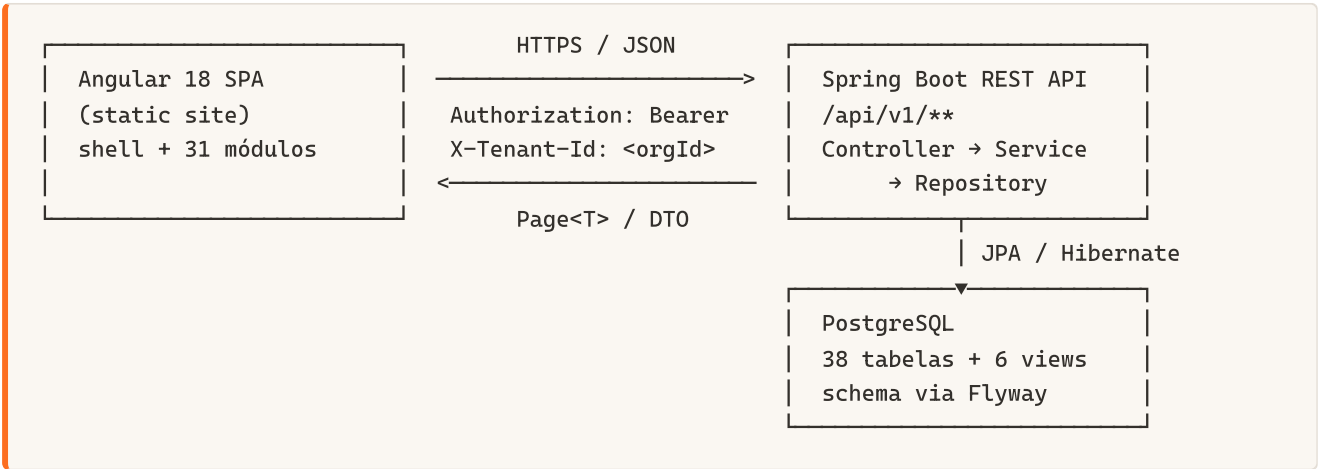
Item	Quantidade
Controllers REST	43
Services de domínio	44 (inclui <code>PipelineEngine</code> , <code>LeadScoringService</code> , <code>CommissionService</code>)
Entidades JPA	38
Migrations Flyway	29 versionadas (V1–V29) + 1 repetível (<code>R__seed_data.sql</code>)
Tabelas	38 + 6 views analíticas
Módulos frontend (rotas lazy)	31 em <code>features/</code> + shell + auth + portal + public-proposal

1.6 Idioma e formatação

Interface 100% pt-BR: moeda BRL, datas `dd/mm/aaaa` , terminologia brasileira. Erros de regra de negócio em português (`BadRequestException`); validações técnicas podem estar em inglês.

2. Arquitetura

2.1 Visão macro



2.2 Backend — organização de pacotes

Raiz: `backend/src/main/java/com/axelcrm/`

Pacote	Responsabilidade
<code>auth/</code>	Módulo vertical: controller, dto, entity, repository, security, service de autenticação
<code>commons/</code>	<code>BaseEntity</code> , <code>Organization</code> , enum <code>Role</code> , exceções e <code>GlobalExceptionHandler</code>
<code>config/</code>	CORS, OpenAPI, auditoria JPA, Flyway programático, criptografia
<code>controller/</code>	Controllers REST dos módulos de negócio (finos)
<code>dto/</code>	Records <code>XxxRequest</code> / <code>XxxResponse</code> com Jakarta Validation
<code>entity/</code>	Entidades JPA e <code>entity/enums/</code>
<code>repository/</code>	Interfaces Spring Data JPA (filtro de tenant no nome)
<code>service/</code>	Regras de negócio (<code>@Transactional</code>)
<code>analytics/</code>	Repositório que lê as views analíticas (<code>AnalyticsViewRepository</code>)

Convenção: `auth` é vertical (fatia por feature); demais módulos são horizontais (por camada). Ao criar código novo, mantenha o padrão do módulo.

2.3 Fluxo de uma requisição autenticada

1. `JwtAuthenticationFilter` lê `Authorization: Bearer <token>`, valida HMAC256 e extrai `userId`, `role`, `organizationId`. Popula `SecurityContextHolder` com `UsernamePasswordAuthenticationToken` (principal = UUID do usuário) e grava `organizationId` como atributo da requisição.
2. `TenantFilter` lê o atributo e coloca no `TenantContext` (`ThreadLocal<UUID>`), limpando no `finally`.
3. **Controller** obtém o tenant via `TenantContext.getOrganizationId()` e repassa ao service.
4. **Service** executa regra e chama repositório sempre filtrando por `organization_id`.
5. **Repository** aplica filtro por convenção de nomes (`...AndOrganization_IdAndDeletedAtIsNull`).
6. Resposta serializada como **DTO record**, nunca entidade JPA.

Ambos os filtros limpam estado no `finally` — obrigatório em pools de threads reutilizadas.

2.4 Padrões de código no backend

Controllers — finos: extraem tenant, delegam ao service, devolvem `ResponseEntity`. Anotados com `@Tag` e `@Operation` para Swagger.

Services — concentram regra, são `@Transactional` e mapeiam entidade → DTO.

DTOs — Java `records` `XxxRequest` / `XxxResponse`, validados com `@Valid` + Jakarta Validation.

Entidades — estendem `BaseEntity`:

- `id` (UUID, gerado pela aplicação)
- `organization` (`@ManyToOne` obrigatório)
- `createdAt` / `updatedAt` (`@CreationTimestamp` / `@UpdateTimestamp`)
- `deletedAt` — soft delete; nenhuma exclusão é física
- Hook `@PrePersist` que preenche organização via `TenantContext` quando não setada

Exclusão sempre lógica: `delete` preenche `deletedAt`; todas as consultas filtram `DeletedAtIsNull`.

2.5 Tratamento de erros

`GlobalExceptionHandler` (`@RestControllerAdvice`) converte em JSON uniforme:

```
{ "error": "mensagem", "status": 400, "timestamp": "2026-08-07T10:00:00" }
```

Exceção	HTTP
<code>ResourceNotFoundException</code>	404
<code>BadRequestException</code>	400
<code>MethodArgumentNotValidException</code>	400 (inclui <code>details</code> com campos)
<code>DataIntegrityViolationException</code>	400
<code>HttpMessageNotReadableException</code>	400
<code>BadCredentialsException</code>	401

`IllegalStateException` lançado por `ProspectService` e `CampaignService` não é tratado — vira 500. Ver pontos de atenção.

2.6 Frontend — organização

Raiz: `frontend/src/app/`

Pasta	Responsabilidade
<code>core/</code>	Guards, interceptors, modelos e serviços transversais
<code>shared/</code>	<code>ListPageComponent</code> , <code>FormDialogComponent</code> , <code>TimelineComponent</code> , <code>BankIcon</code>
<code>shell/</code>	Layout autenticado: sidebar, toolbar, notificações, tema, cronômetro global
<code>features/</code>	Um diretório por módulo com <code>*.routes.ts</code> e componentes lazy-loaded

Todos os componentes são **standalone**; rotas usam `loadChildren` / `loadComponent` (cada módulo = chunk separado).

2.7 Camada de dados do frontend

`BaseService<T>` centraliza `getPage`, `getById`, `create`, `update`, `delete` sobre `environment.apiUrl`. Serviços de módulo herdam e fixam o path.

Dois interceptors em `app.config.ts`:

- `authInterceptor` — injeta `Authorization: Bearer <token>`; em 401 faz logout e redireciona a `/login`
- `tenantInterceptor` — injeta `X-Tenant-Id` (informativo; backend não o usa — tenant vem do JWT)

3. Segurança e Multi-Tenancy

3.1 Autenticação

API **stateless** (`SessionCreationPolicy.STATELESS`), sem sessão HTTP e CSRF desabilitado — coerente com SPA + token.

Token JWT

Gerado por `JwtUtil` com **HMAC256**:

Claim	Conteúdo
<code>userId</code>	UUID do usuário
<code>email</code>	E-mail
<code>role</code>	Papel (<code>ADMIN</code> , <code>SALES</code> ...)
<code>organizationId</code>	UUID da organização (tenant)
<code>iat</code> / <code>exp</code>	Emissão / expiração

Validade padrão **86.400.000 ms (24 h)**, configurável por `JWT_EXPIRATION` (ver atenção sobre `jwt.expiration` vs `jwt.expiration-ms`).

Registro e login

- `POST /api/v1/auth/register` — cria **nova organização** + primeiro usuário `ADMIN`. Não existe convite para org existente via esse endpoint; usuários adicionais via `/api/v1/users`.
- `POST /api/v1/auth/login` — valida e-mail/senha (BCrypt), recusa inativo, devolve `LoginResponse`: `token`, `userId`, `userName`, `email`, `role`, `organizationId`, `organizationName`.

Senhas com `BCryptPasswordEncoder`.

3.2 Rotas públicas

Declaradas em `SecurityConfig.PUBLIC_PATHS`:

```
/ /api/v1/auth/** /api/v1/proposals/public/** /swagger-ui/** /swagger-ui.html
/v3/api-docs /v3/api-docs/** /actuator/health /actuator/info /error
```

`/error` é público para não mascarar erros como falha de CORS na SPA. Demais rotas exigem token válido.

3.3 Isolamento multi-tenant

Isolamento **em nível de aplicação** (sem RLS nem schema por tenant). Três mecanismos combinados:

1. **TenantContext** — `ThreadLocal<UUID>` populado por `TenantFilter` a partir do JWT, limpo ao fim da requisição.
2. **Passagem explícita** — `organizationId` do controller ao service ao repositório em toda operação.
3. **Consultas filtradas** — métodos como `findByIdAndOrganization_IdAndDeletedAtIsNull(UUID id, UUID orgId)`.

`BaseEntity.prePersist()` atribui organização via `TenantContext` quando a entidade é persistida sem organização explícita.

Regra de ouro: todo repositório novo precisa filtrar por `organization_id`. Sem isso, há vazamento entre tenants.

3.4 Autorização por papel

Enum `Role` tem 7 valores; papel entra no token como authority. Frontend: `adminGuard` bloqueia `/users` e `/integrations`. Backend: **nenhum** `@PreAuthorize` — qualquer usuário autenticado da organização pode chamar qualquer endpoint privado. Guard de frontend não é controle de acesso.

3.5 CORS

`CorsConfig` permite `allowedOriginPatterns: *`, métodos `GET, POST, PUT, DELETE, PATCH, OPTIONS`, qualquer header, `allowCredentials: true`, cache preflight 1 h, expõe `Authorization` e `X-Tenant-Id`.

Atenção: não deve ir assim a produção — restringir por perfil.

3.6 Armazenamento de sessão no cliente

`AuthService` grava em `localStorage`:

Chave	Conteúdo
<code>crm_token</code>	JWT
<code>crm_user</code>	JSON com id, nome, e-mail, papel e organização
<code>crm_tenant</code>	<code>organizationId</code>

Logout remove as três chaves e redireciona a `/login`. Sem refresh token: expirado o JWT, próxima chamada retorna 401 e o interceptor força login.

3.7 LGPD

`LgpdService` / `LgpdController` (`/api/v1/lgpd`):

Endpoint	Função
<code>POST /consent</code>	Registra/atualiza consentimento por e-mail + tipo (<code>lgpd_consent</code>)
<code>GET /export</code>	Portabilidade: exporta dados pessoais (leads, clientes, contatos, consentimentos)
<code>DELETE /forget</code>	Esquecimento: exclusão/anonimização do titular

Consentimento único por e-mail + tipo dentro da organização.

3.8 Trilha de auditoria

`AuditLogService` grava em `audit_logs` (organização, usuário, ação, entidade, valor anterior/novo).
Ações: `STAGE_TRANSITION`, `DEAL_LOST`, `DEAL_REOPENED`, `CONVERT`. Leitura em `/api/v1/audit-logs`.

4. Modelo de Dados

O schema é gerenciado **exclusivamente pelo Flyway** (`backend/src/main/resources/db/migration/`).
Hibernate com `ddl-auto: none` nunca altera o banco.

4.1 Convenções comuns

Toda tabela de negócio herda de `BaseEntity`:

Coluna	Tipo	Observação
<code>id</code>	<code>UUID</code>	PK gerada pela aplicação
<code>organization_id</code>	<code>UUID</code>	FK obrigatória para <code>organizations</code>
<code>created_at</code>	<code>TIMESTAMP</code>	No insert
<code>updated_at</code>	<code>TIMESTAMP</code>	A cada update
<code>deleted_at</code>	<code>TIMESTAMP</code>	Soft delete: <code>NULL</code> = ativo

Exceções: `organizations` e `users` (users referencia organização, organizations é raiz).

4.2 Tabelas por domínio

Núcleo

Tabela	Descrição
<code>organizations</code>	Tenant: nome, domínio único, documento, contato, endereço, <code>active</code>
<code>users</code>	Usuário: e-mail único global, senha BCrypt, papel, avatar, <code>active</code>
<code>audit_logs</code>	Trilha de auditoria
<code>notifications</code>	Notificações lidas/não lidas
<code>integrations</code>	Config de integrações externas
<code>google_tokens</code>	Tokens OAuth2 Google por usuário
<code>lgpd_consent</code>	Consentimentos LGPD

Comercial (CRM)

Tabela	Descrição
<code>prospects</code>	Topo do funil, antes de virar lead
<code>partners</code>	Parceiros / indicadores
<code>leads</code>	Lead com origem, estágio, score, valor estimado
<code>lead_notes</code>	Anotações do lead
<code>clients</code>	Cliente convertido (status, segmento, responsável)
<code>client_notes</code>	Anotações do cliente
<code>client_attachments</code>	Anexos do cliente
<code>contacts</code>	Pessoas de contato vinculadas a clientes
<code>pipelines</code>	Funil de vendas
<code>pipeline_stages</code>	Estágios ordenados por <code>position</code>
<code>deals</code>	Negócio: valor, pipeline, estágio, cliente, responsável
<code>deal_stage_history</code>	Histórico de permanência por estágio (duração/motivo)
<code>campaigns</code>	Campanhas de marketing
<code>campaign_recipients</code>	Destinatários por campanha
<code>messages</code>	Comunicações (e-mail, WhatsApp, SMS, ligação, Instagram)

Operações

Tabela	Descrição
<code>proposals</code>	Proposta (código, token público, rateio de comissões)
<code>proposal_items</code>	Itens da proposta
<code>contracts</code>	Contratos
<code>projects</code>	Projeto de entrega (orçamento, custo, dados periciais)
<code>products</code>	Catálogo de produtos/serviços
<code>tasks</code>	Tarefas
<code>calendar_events</code>	Eventos de agenda
<code>legal_processes</code>	Processos jurídicos (nº CNJ)
<code>documents</code>	Documentos
<code>support_tickets</code>	Tickets de suporte
<code>time_entries</code>	Apontamento de horas
<code>whatsapp_*</code>	Inbox WhatsApp (ver V29)

Financeiro

Tabela	Descrição
<code>invoices</code>	Fatura: número único, emissão, vencimento, pagamento, subtotal, impostos, desconto, total
<code>financial_transactions</code>	Lançamentos: entrada, saída, transferência, estorno, ajuste
<code>chart_of_accounts</code>	Plano de contas hierárquico
<code>bank_accounts</code>	Contas bancárias
<code>commissions</code>	Comissões apuradas
<code>commission_rules</code>	Regras de cálculo

4.3 Entidades principais (resumo)

Client: `name`, `email`, `phone`, `document`, `companyName`, `website`, `address`, `city`, `state`, `zipCode`, `country`, `industry`, `notes`, `active`, `status` (`ClientStatus`), `serviceType`, `assignedTo`, `contacts`.

Lead: `name`, `email`, `phone`, `company`, `position`, `source` (`LeadSource`), `stage` (`LeadStage`), `notes`, `score`, `estimatedValue`, `lastContactAt`, `convertedAt`, `assignedTo`, `convertedClient`, `partner`.

Deal: `title`, `description`, `value`, `pipeline`, `stage`, `client`, `contact`, `assignedTo`, `expectedCloseDate`, `closedAt`, `won` (três estados: `null` = aberto, `true` = ganho, `false` = perdido).

Proposal: `proposalCode`, `publicToken` (UUID), `title`, `description`, `status` (`ProposalStatus`), `issueDate`, `validUntil`, `totalAmount`, `discountAmount`, `approvedAt`, `client`, `assignedTo`, `items`, `deal`, `partner`, `rateio` (`captureUser/Rate`, `sellerUser/Rate`, `collaboratorUser/Rate`, `partnerRate`), bloco jurídico-pericial (V28): `lawyerContact` (FK `contacts`), `lawyerName`, `referralSource` (`LeadSource`), `expertUser`, `technicalManagerUser`, `project`.

- `lawyerContact` e `lawyerName` são mutuamente exclusivos (com contato, nome livre é zerado; leitura devolve nome do contato via `resolveLawyerName()`).
- `Proposal.project` ↔ `Project.sourceProposalId` (bidirecional por caminhos distintos).

Project: `name`, `description`, `startDate`, `endDate`, `budget`, `cost`, `status` (string livre, padrão `PLANEJAMENTO`), `client`, `manager`, `sourceProposalId`, `legalProcess`, `cnjNumber`, `expertType`, `paymentStatus`, `deliveryDeadline`.

Invoice: `invoiceNumber` (único), `client`, `contract`, `issueDate`, `dueDate`, `paidDate`, `status` (string `DRAFT`), `subtotal`, `taxAmount`, `discountAmount`, `total`, `paidAmount`, `paymentMethod`, `notes`.

4.4 Enums

Enum	Valores
<code>LeadSource</code>	<code>WEBSITE</code> , <code>SOCIAL_MEDIA</code> , <code>REFERRAL</code> , <code>EMAIL</code> , <code>PHONE</code> , <code>EVENT</code> , <code>ADVERTISEMENT</code> , <code>PARTNER</code> , <code>OTHER</code>
<code>LeadStage</code>	<code>NEW</code> , <code>CONTACTED</code> , <code>QUALIFIED</code> , <code>PROPOSAL</code> , <code>NEGOTIATION</code> , <code>CONVERTED</code> , <code>LOST</code> , <code>ARCHIVED</code>
<code>ProspectStage</code>	<code>PROSPECTING</code> , <code>CONTACTED</code> , <code>WAITING_REPLY</code>
<code>ClientStatus</code>	<code>NEW</code> , <code>CONTACTED</code> , <code>QUALIFIED</code> , <code>WAITING_RESPONSE</code> , <code>CLOSED_WON</code> , <code>CLOSED_LOST</code> , <code>DISCARDED</code> , <code>ONBOARDING</code> , <code>ACTIVE</code> , <code>INACTIVE</code>
<code>ProposalStatus</code>	<code>DRAFT</code> , <code>SENT</code> , <code>VIEWED</code> , <code>NEGOTIATING</code> , <code>ACCEPTED</code> , <code>REJECTED</code> , <code>EXPIRED</code>
<code>TaskStatus</code>	<code>PENDING</code> , <code>IN_PROGRESS</code> , <code>BLOCKED</code> , <code>COMPLETED</code> , <code>CANCELLED</code>
<code>TransactionType</code>	<code>INCOME</code> , <code>EXPENSE</code> , <code>TRANSFER</code> , <code>REFUND</code> , <code>ADJUSTMENT</code>
<code>ChartOfAccountType</code>	<code>RECEITA</code> , <code>DESPESA</code> , <code>ATIVO</code> , <code>PASSIVO</code>
<code>CampaignType</code>	<code>EMAIL</code> , <code>SOCIAL</code> , <code>PPC</code> , <code>SEO</code> , <code>EVENT</code> , <code>WEBINAR</code> , <code>DIRECT_MAIL</code> , <code>OTHER</code>
<code>MessageChannel</code>	<code>EMAIL</code> , <code>WHATSAPP</code> , <code>SMS</code> , <code>CALL</code> , <code>INSTAGRAM</code>
<code>MessageDirection</code>	<code>INBOUND</code> , <code>OUTBOUND</code>
<code>ContactType</code>	<code>LAWYER</code> , <code>JUDGE</code> , <code>CLIENT</code> , <code>TECHNICAL_ASSISTANT</code> , <code>OTHER</code>
<code>Role</code>	<code>SUPER_ADMIN</code> , <code>ADMIN</code> , <code>MANAGER</code> , <code>SALES</code> , <code>SUPPORT</code> , <code>USER</code> , <code>VIEWER</code>

`Project.status`, `Invoice.status`, `Contract.status` são strings livres sem validação.

4.5 Views analíticas

Criadas em `V25__create_analytics_views.sql`, lidas por `AnalyticsViewRepository`:

View	Conteúdo
<code>vw_entity_counts</code>	Contagens por entidade (cards do dashboard)
<code>vw_deal_pipeline</code>	Distribuição de negócios por pipeline/estágio
<code>vw_lead_analytics</code>	Funil e desempenho de leads
<code>vw_proposal_analytics</code>	Métricas de propostas
<code>vw_monthly_financial</code>	Série mensal receitas/despesas
<code>vw_project_profitability</code>	Rentabilidade por projeto (orçamento × custo)

4.6 Migrations

Nomenclatura `V<n>__descricao.sql` (versionadas) e `R__seed_data.sql` (repetível). Linha do tempo:

Faixa	Escopo
V1	Organizações e usuários
V2–V8	Leads, clientes, pipelines, negócios, propostas, projetos, tarefas, agenda, financeiro, campanhas, notificações, suporte
V9–V17	Refinamento leads/clientes, LGPD, mensagens, prospects, parceiros, status
V18–V24	Token público de proposta, plano de contas, processos jurídicos/cronômetro, contratos, produtos, faturas, documentos
V25–V26	Views analíticas, vínculo propostas-negócios e histórico de estágio
V27–V28	Tokens Google; campos jurídicos/periciais da proposta
V29	WhatsApp inbox

5. API REST

Base `/api/v1`. Swagger em `/swagger-ui.html`; OpenAPI em `/v3/api-docs`.

5.1 Convenções

Autenticação: `Authorization: Bearer <jwt>` em todo endpoint privado; tenant vem do claim — não enviar no body/URL.

CRUD padrão (5 operações):

Método	Path	Resposta
GET	/api/v1/<recurso>	Page<XxxResponse>
GET	/api/v1/<recurso>/{id}	XxxResponse
POST	/api/v1/<recurso>	XxxResponse (200)
PUT	/api/v1/<recurso>/{id}	XxxResponse
DELETE	/api/v1/<recurso>/{id}	204 No Content (soft delete)

Paginação (Spring Data):

```
GET /api/v1/clients?page=0&size=10&sort=name,asc
```

Resposta: `content`, `totalElements`, `totalPages`, `number`, `size`, `first`, `last`.

Erros (GlobalExceptionHandler):

```
{
  "error": "Erro de validação de dados",
  "details": ["email: não deve estar em branco"],
  "status": 400,
  "timestamp": "2026-08-07T10:00:00"
}
```

5.2 Autenticação e usuários

Método	Endpoint	Descrição
POST	/auth/register	Cria organização + usuário <code>ADMIN</code> e devolve token
POST	/auth/login	Autentica e devolve token
GET	/users, /users/{id}	Lista/consulta usuários da org
GET	/users/me	Dados do usuário autenticado
POST PUT DELETE	/users, /users/{id}	Gestão de usuários

5.3 CRM

Recurso	Base	Endpoints adicionais
Prospects	/prospects	POST /{id}/promote → promove a lead
Parceiros	/partners	—
Leads	/leads	POST /{id}/convert → cliente; POST /{id}/recalculate-score
Notas de lead	/leads/{leadId}/notes	DELETE /{noteId}
Timeline lead	/leads/{leadId}/timeline	Somente leitura
Cientes	/clients	—
Notas cliente	/clients/{clientId}/notes	DELETE /{noteId}
Anexos cliente	/clients/{clientId}/attachments	GET /{attachmentId}/download, DELETE
Timeline cliente	/clients/{clientId}/timeline	Somente leitura
Contatos	/contacts	—
Pipelines	/pipelines	GET /{id}/stages, POST /stages, PUT /stages/{id}, DELETE /stages/{id}
Negócios	/deals	POST /{id}/transition, POST /{id}/mark-lost, POST /{id}/reopen, POST /{id}/convert-to-project
Comunicações	/communications	GET /lead/{leadId}, GET /client/{clientId}
Campanhas	/campaigns	POST /{id}/send
WhatsApp	/whatsapp/*	Inbox, QR, vínculo (ver 5.7)

5.4 Operações

Recurso	Base	Endpoints adicionais
Propostas	/proposals	GET /public/{token} (público), GET /{id}/pdf, GET /public/{token}/pdf (público), POST /{id}/convert-to-project
Contratos	/contracts	—
Projetos	/projects	—
Produtos	/products	—
Tarefas	/tasks	—

Recurso	Base	Endpoints adicionais
Agenda	/calendar-events	—
Processos jurídicos	/legal-processes	GET /search
Documentos	/documents	—
Tickets	/support-tickets	—
Apontamento horas	/time-entries	—

Proposta aceita `ProposalRequest` com bloco jurídico-pericial: `lawyerContactId`, `lawyerName`, `referralSource`, `expertUserId`, `technicalManagerUserId`, `projectId`. Resposta devolve `lawyerContactId`, `lawyerName` (resolvido), `referralSource`, `expertUser`, `technicalManagerUser`, `projectId`, `projectName`.

5.5 Financeiro

Recurso	Base	Endpoints adicionais
Faturas	/invoices	GET /{id}/pdf, GET /report/pdf
Transações	/financial-transactions	—
Plano de contas	/chart-of-accounts	GET /tree, POST /import
Contas bancárias	/bank-accounts	—
Comissões	/commissions	—
Regras comissão	/commission-rules	—
Relatórios financeiros	/financial-reports	GET /cash-flow, GET /income-statement
Relatórios	/reports	GET /dre, GET /dfc

5.6 Dashboard e analytics

Método	Endpoint	Conteúdo
GET	/dashboard/summary	Resumo consolidado
GET	/dashboard/counts	Contadores por entidade
GET	/dashboard/sales	Métricas de vendas
GET	/dashboard/leads	Funil de leads

Método	Endpoint	Conteúdo
GET	/dashboard/financial-trend	Série mensal
GET	/dashboard/projects	Rentabilidade projetos
GET	/dashboard/proposals	Métricas propostas
GET	/analytics/dashboard	Agregação direta das views

5.7 WhatsApp

Documentado em `docs/specs/whatsapp-inbox.md` e frontend `whatsapp.service.ts`:

- Conexão via QR code, status de vínculo, inbox de conversas e envio de mensagens.
- Endpoints sob `/api/v1/whatsapp/**` (ver Swagger). Migration `V29__whatsapp_inbox.sql`.

5.8 Transversais

Recurso	Base	Observação
Notificações	/notifications	PUT /{id}/read marca como lida
Audit logs	/audit-logs	Somente leitura
LGPD	/lgpd	POST /consent, GET /export, DELETE /forget
Integrações	/integrations	CRUD + /google/status, /google/connect, /google/disconnect, /google/calendar, /ai/generate

`/integrations/google/*` e `/ai/generate` no controller devolvem **mocks** (não usam `GoogleIntegrationService`). Ver pontos de atenção.

5.9 Endpoints públicos

Proposta pública: GET `/api/v1/proposals/public/{token}` e GET `.../public/{token}/pdf` localizam por `publicToken` (UUID). Renderizado em `/public/proposals/:token` na SPA. Token não expira, sem segundo fator.

Saúde: GET `/actuator/health` e `/actuator/info` (`show-details: never`).

6. Frontend

SPA Angular 18 com componentes standalone, Material 18 e Chart.js. Todo conteúdo é lazy-loaded por módulo.

6.1 Estrutura de rotas (`app.routes.ts`)

```
/login                público
/register             público
/public/proposals/:token público - proposta
/portal/**            portais cliente/parceiro
/                     ShellComponent - authGuard
├── dashboard         (padrão)
├── profile
├── 28+ módulos de negócio (lazy)
├── users             + adminGuard
└── integrations      + adminGuard
**                  redirect → /
```

Módulos lazy: `products`, `calendar`, `clients`, `partners`, `prospects`, `leads`, `contacts`, `deals`, `whatsapp`, `pipelines`, `projects`, `tasks`, `proposals`, `contracts`, `campaigns`, `tickets`, `invoices`, `transactions`, `bank-accounts`, `time-entries`, `commissions`, `chart-of-accounts`, `reports`, `legal-processes`, `documents`, `users`, `integrations`.

6.2 Shell

`ShellComponent`: sidebar recolhível, toolbar com breadcrumbs, notificações com polling, alternância tema claro/escuro, menu usuário e cronômetro global (`TimerComponent`) para `time-entries`. Tour de onboarding com passos guiados.

Navegação agrupada:

Seção	Itens
Topo	Dashboard
CRM	Prospecção, Agenda, Parceiros, Clientes, Contatos, Leads, Negócios, Pipelines, WhatsApp
Operações	Projetos, Produtos, Contratos, Processos, Tarefas, Propostas, Campanhas, Tickets, Documentos
Financeiro	Faturamento, Transações, Plano de Contas, Relatórios, Contas Bancárias, Horas, Comissões
Admin	Usuários, Integrações

Rótulo no shell: "IBCAPP CRM"; nome oficial **Axel CRM** (`PRODUCT.md`).

6.3 Componentes compartilhados

`ListPageComponent` — base de listagens, dirigido por `@Input` / `@Output`, sem conhecimento de domínio:

Inputs: `title`, `columns: ColumnDef[]` (`{key, label}`), `data`, `totalElements`, `pageSize`, `loading`, `error`, `emptyMessage/Icon/ActionLabel`, `kpis: KpiDef[]`. Outputs: `pageChange`, `sortChange`, `add`, `edit`, `remove`, `view`, `retry`. Coluna `actions` anexada automaticamente.

FormDialogComponent — diálogo genérico de formulário dirigido por configuração de campos (criar/editar).

TimelineComponent — linha do tempo de leads/clientes via **TimelineService**.

Outros: **BankIconComponent** (ícones de bancos brasileiros).

6.4 Camada core

Arquivo	Função
<code>core/guards/auth.guard.ts</code>	Exige token; redirect <code>/login</code>
<code>core/guards/admin.guard.ts</code>	Exige <code>ADMIN</code> / <code>SUPER_ADMIN</code> ; redirect <code>/dashboard</code>
<code>core/interceptors/auth.interceptor.ts</code>	Injeta Bearer; em 401 faz logout
<code>core/interceptors/tenant.interceptor.ts</code>	Injeta <code>X-Tenant-Id</code>
<code>core/services/auth.service.ts</code>	Login, registro, logout, <code>BehaviorSubject</code> do usuário
<code>core/services/base.service.ts</code>	CRUD HTTP genérico
<code>core/services/client-detail.service.ts</code>	Agregação detalhe cliente
<code>core/services/lead-detail.service.ts</code>	Agregação detalhe lead
<code>core/services/timeline.service.ts</code>	Timeline
<code>core/models/models.ts</code>	<code>Page<T></code> , <code>User</code> , DTOs compartilhados

BaseService<T> :

```
getPage(path, page=0, size=10, sort='id,asc'): Observable<Page<T>>
getById(path, id): Observable<T>
create(path, body): Observable<T>
update(path, id, body): Observable<T>
delete(path, id): Observable<void>
```

6.5 Telas com layout próprio

Tela	Particularidade
<code>dashboard</code>	KPIs, gráficos Chart.js, funil, tarefas e comissões recentes
<code>clients/client-detail</code>	Abas notas, anexos, contatos, timeline
<code>leads/lead-detail</code>	Score, timeline, comunicações
<code>projects/project-detail</code>	Dados de entrega e financeiro
<code>proposals/public-proposal</code>	Visualização pública por token, fora do shell

Tela	Particularidade
<code>portal/*</code>	Portais externos
<code>reports</code>	DRE e DFC
<code>integrations</code>	Configuração (admin)
<code>profile</code>	Perfil
<code>whatsapp-inbox</code>	Inbox com QR dialog e link dialog

6.6 Configuração de ambiente

`src/environments/environment.ts` (prod) e `environment.development.ts` (local) expõem `apiUrl`. No Render, build substitui `__API_URL__` por `API_URL` via `sed` em `render.yaml`.

```
export const environment = { production: false, apiUrl: 'http://localhost:8080/api/v1' };
```

6.7 Design system

Tokens em `DESIGN.md`: tema escuro padrão (`:root`), claro em `[data-theme="light"]`, laranja `#FC6E20` como cor de ação, tipografia Outfit (estrutura) + Inter (dados), Material Icons Round, raios e espaçamentos padronizados.

7. Regras de Negócio

Este capítulo descreve fluxos além de CRUD. Demais módulos seguem CRUD padrão do cap. 5.

7.1 Funil comercial

```

Prospect —promote→ Lead —convert→ Client
                        |
                        └→ Deal —proposta aceita→ Project → Invoice

```

Prospect → Lead

`POST /api/v1/prospects/{id}/promote` (`ProspectService.promoteToLead`): copia nome, e-mail, telefone, empresa, origem e anotações para novo `Lead` em `NEW`; prospect aponta para `convertedLead` e `convertedAt`; já convertido não pode ser promovido de novo (400).

Lead → Client

`POST /api/v1/leads/{id}/convert` (`LeadConversionService.convertLeadToClient`): cria `Client` (nome, e-mail, telefone, empresa → `companyName`, anotações, responsável), marca lead `CONVERTED` , grava `convertedAt` e `convertedClient`, audita `CONVERT` . **Idempotente**: se já `CONVERTED` com cliente vinculado, devolve cliente existente.

7.2 Lead scoring

`POST /api/v1/leads/{id}/recalculate-score` (`LeadScoringService.recalculate`): soma 4 componentes, limita 0–100.

1. **Valor estimado:** 1 ponto a cada R\$ 100 (floor).
2. **Origem:**

Origem	Pontos
REFERRAL	30
EVENT	25
WEBSITE	20
SOCIAL_MEDIA	15
EMAIL	10
PHONE	5
demais	8

3. **Contato recente** (`lastContactAt`): ≤ 3 dias +20; ≤ 7 dias +10; ≤ 30 dias +5.
4. **Recência cadastro** (`createdAt`): ≤ 7 dias +15; ≤ 30 dias +10; ≤ 90 dias +5.

Sob demanda — sem job periódico.

7.3 Motor de pipeline

`PipelineEngine` centraliza mudanças de estágio e é única via que grava `deal_stage_history`.
Transacional + auditoria.

Transição de estágio

`POST /api/v1/deals/{id}/transition` com `targetStageId` e `reason` opcional. Regras:

1. Negócio já **ganho** não pode mudar estágio.
2. Negócio **perdido** precisa ser reaberto antes.
3. Estágio destino precisa pertencer ao **mesmo pipeline**.
4. Movimento **regressivo** (posição destino < atual) exige `reason`, senão 400.

Execução: fecha registro aberto em `deal_stage_history` (`leftAt`, `durationSeconds`), move negócio, se destino é último estágio marca `won=true` + `closedAt`, abre novo histórico (`enteredAt`, motivo, autor), audita `STAGE_TRANSITION`.

Marcar como perdido

`POST /api/v1/deals/{id}/mark-lost`: recusa já ganhos; fecha histórico, `won=false`, `closedAt`, audita `DEAL_LOST`.

Reabrir

`POST /api/v1/deals/{id}/reopen`: recusa não-fechados; zera `won` / `closedAt`, abre novo histórico no estágio atual, audita `DEAL_REOPENED`.

Duração por estágio

Cada linha de `deal_stage_history` guarda `enteredAt`, `leftAt`, `durationSeconds` — permite medir tempo médio por estágio e gargalos.

7.4 Propostas

Cálculo do total

Ao criar/atualizar: total = soma dos itens – `discountAmount`. Itens persistidos em `proposal_items`.

Campos jurídicos e periciais

Campo	Regra
Advogado vinculado	FK <code>contacts</code> da org; omitir limpa vínculo; inexistente → 404
Nome do advogado	Texto livre gravado apenas sem contato; com contato, zerado e leitura devolve nome do contato
Origem indicação	<code>LeadSource</code> ; quem indicou é <code>partner</code> (taxa de comissão)
Perito responsável	FK <code>users</code> ; não entra no rateio
Responsável técnico	FK <code>users</code> ; não entra no rateio
Projeto vinculado	FK <code>projects</code> da org

Ausência no request limpa valor atual (convenção da proposta).

Aprovação e ações automáticas

Quando status → `ACCEPTED`, grava `approvedAt` e dispara `triggerPostApprovalActions`:

- Avança negócio vinculado** — move para último estágio do pipeline com motivo "Aprovação automática via Proposta comercial" (como é estágio final, marca como ganho).
- Cria projeto** — se já tem projeto vinculado, nada faz; senão, se não existir projeto com `sourceProposalId` igual, cria com nome `Projeto: <título>`, descrição/cliente/responsável herdados, `startDate` = hoje, `endDate` = hoje+3 meses, `budget` = total da proposta, `cost` = 0, `status` = `PLANEJAMENTO`. Projeto criado vira vínculo da proposta.

Proteções contra duplicação: vínculo direto (`Proposal.project`) e checagem por `sourceProposalId`.

Conversão manual

`POST /api/v1/proposals/{id}/convert-to-project`: mesma criação; 400 se não `ACCEPTED` ("A proposta precisa estar aprovada...") ou já tem projeto vinculado ("Esta proposta já possui um projeto vinculado.").

`POST /api/v1/deals/{id}/convert-to-project` cria projeto a partir do negócio.

Link público

Cada proposta recebe `publicToken` (UUID). `GET /api/v1/proposals/public/{token}` e `.../pdf` sem autenticação; SPA em `/public/proposals/:token`. Token não expira, sem segundo fator.

PDF

`ProposalService` gera PDF via OpenPDF (rotas autenticada e pública); `InvoiceService` faz o mesmo para faturas.

7.5 Comissões

Manual/por regra: ao criar/atualizar comissão, se `amount` não informado e houver `CommissionRule` aplicável, `valor = dealValue × percentage`.

Multinível a partir da proposta: `calculateCommissionsForTransaction` distribui a partir de transação financeira entre até 4 papéis definidos na proposta:

Papel	Beneficiário	Percentual
<code>CAPTURE</code>	<code>captureUser</code>	<code>captureRate</code>
<code>SELLER</code>	<code>sellerUser</code>	<code>sellerRate</code>
<code>PARTNER</code>	<code>partner</code>	<code>partnerRate</code>
<code>COLLABORATOR</code>	<code>collaboratorUser</code>	<code>collaboratorRate</code>

Cada comissão = `valor transação × taxa`; só criada se beneficiário existe e taxa >0. Apuração conforme dinheiro entra, não no fechamento.

`payCommission` marca como paga.

7.6 Apontamento de horas

`TimeEntryService`: aceita `durationMinutes` explícito; se ausente mas `startTime` / `endTime` preenchidos, calcula diferença. Shell tem `TimerComponent` global.

7.7 Campanhas

`POST /api/v1/campaigns/{id}/send`: monta `campaign_recipients` a partir de **todos** os leads e clientes da org com e-mail/telefone, marca como enviada; já enviada não pode ser reenviada. **Simulação:** destinatários gravados, nenhuma mensagem disparada (sem provedor). Ver pontos de atenção sobre `findAll()` sem filtro.

7.8 Plano de contas

`GET /api/v1/chart-of-accounts/tree` devolve hierarquia; `POST /import` importa plano padrão. Tipos `RECEITA`, `DESPESA`, `ATIVO`, `PASSIVO`.

7.9 Relatórios financeiros

Endpoint	Relatório
<code>/financial-reports/cash-flow</code>	Fluxo de caixa
<code>/financial-reports/income-statement</code>	Demonstração resultado
<code>/reports/dre</code>	DRE
<code>/reports/dfc</code>	DFC

Números do dashboard vêm das views analíticas do cap. 4.

8. Ambiente e Deploy

8.1 Pré-requisitos

Ferramenta	Versão
Java (JDK)	21
Node.js	20+
PostgreSQL	16+
Maven	3.9+ (sem wrapper <code>mvnw</code> no repo)

8.2 Execução local

Banco:

```
createdb axelcrm
# application.yml aponta para jdbc:postgresql://localhost:5440/axelcrm (user postgres/123456)
```

Backend:

```
cd backend
mvn spring-boot:run
# API em http://localhost:8080 - Flyway aplica migrations automaticamente
# logs verbosos:
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

Frontend:

```
cd frontend
npm install
npm start # ng serve
# App em http://localhost:4200
```

Config frontend (`src/environments/environment.ts`):

```
export const environment = { production: false, apiUrl: 'http://localhost:8080/api/v1' };
```

Primeiro acesso: `/register` cria organização + usuário `ADMIN`.

8.3 Documentação da API em execução

Recurso	URL local
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI JSON	<code>http://localhost:8080/v3/api-docs</code>
Health	<code>http://localhost:8080/actuator/health</code>

8.4 Perfis de configuração

Arquivo	Uso
<code>application.yml</code>	Padrão local: PG localhost, JWT dev, logs security/web DEBUG
<code>application-dev.yml</code>	Acrescenta <code>show-sql</code> , SQL formatado, log DEBUG <code>com.axelcrm</code>
<code>application-render.yml</code>	Produção Render: conexão/secredo via env, logs INFO

Fixos em todos os perfis: `spring.jpa.hibernate.ddl-auto: none`, Flyway `enabled: true`, `baseline-on-migrate: true`.

8.5 Variáveis de ambiente

Backend:

Variável	Perfil	Descrição
<code>SPRING_PROFILES_ACTIVE</code>	todos	Perfil ativo (<code>render</code> em produção)
<code>PORT</code>	render	Porta HTTP (8080)
<code>DB_HOST</code> <code>DB_PORT</code> <code>DB_NAME</code> <code>DB_USER</code> <code>DB_PASSWORD</code>	render	Conexão PG (injetadas via <code>fromDatabase</code> do Blueprint)

Variável	Perfil	Descrição
<code>JWT_SECRET</code>	render	Segredo HMAC256 — obrigatório, manual
<code>JWT_EXPIRATION</code>	render	Validade token ms (padrão 86400000)
<code>google.client-id</code> / <code>google.client-secret</code> / <code>google.redirect-uri</code>	opcional	OAuth2 Google (<code>GoogleIntegrationService</code>)

`application.yml` traz `jwt.secret` dev embutido — nunca usar em produção.

Frontend:

Variável	Descrição
<code>API_URL</code>	URL base da API; substitui placeholder <code>__API_URL__</code> no build

8.6 Migrations

Local `backend/src/main/resources/db/migration/`. Nomenclatura `V<n>__descricao.sql` (versionadas) e `R__nome.sql` (repetíveis). `FlywayMigrationConfig` implementa `CommandLineRunner` e executa `flyway.repair()` + `flyway.migrate()` na subida — `repair` corrige checksums divergentes (risco: edições em migrations aplicadas passam despercebidas). **Nunca edite migration aplicada; crie nova.**

Ao criar migration:

1. Use próximo número livre (última V29).
2. Inclua `organization_id UUID NOT NULL FK organizations`.
3. Inclua `created_at`, `updated_at`, `deleted_at`.
4. Índice em `organization_id`.

8.7 Deploy no Render

`render.yaml` Blueprint provisiona 3 recursos:

Recurso	Nome	Tipo
Banco	<code>axel-crm-db</code>	PostgreSQL free
API	<code>axel-crm-api</code>	Web service Docker, <code>rootDir: backend</code>
SPA	<code>axel-crm-app</code>	Static site, <code>rootDir: frontend</code>

Credenciais do banco injetadas automaticamente no serviço API. `JWT_SECRET` com `sync: false` — preencher manualmente no painel, sem ele a app não sobe.

Build frontend:

```
npm ci && sed -i "s|__API_URL__|$API_URL|g" src/environments/environment.ts \
&& npm run build -- --configuration production
```

Diretório publicado `dist/crm-axel-frontend/browser` com rewrite `/*` → `/index.html` (necessário ao roteamento Angular).

URLs esperadas:

Serviço	URL
API	<code>https://axel-crm-api.onrender.com</code>
Swagger UI	<code>https://axel-crm-api.onrender.com/swagger-ui.html</code>
Frontend	<code>https://axel-crm-app.onrender.com</code>

Docker backend (`backend/Dockerfile`): multi-stage `maven:3.9-eclipse-temurin-21` compila JAR com `-DskipTests`; `eclipse-temurin:21-jre` executa artefato, expõe 8080.

9. Guia de Desenvolvimento

9.1 Como adicionar um módulo CRUD completo

Use `Client` como referência viva — exemplo mais limpo.

Backend:

1. **Migration** `V<n>__create_<tabela>.sql` com `id UUID PK`, `organization_id UUID NOT NULL FK`, colunas de domínio, `created_at/updated_at/deleted_at`, índice em `organization_id`.
2. **Entidade** em `entity/` estendendo `BaseEntity`:

```
@Entity @Table(name = "widgets")
@Data @EqualsAndHashCode(callSuper = true)
public class Widget extends BaseEntity {
    @Column(nullable = false) private String name;
}
```

3. **Repositório** em `repository/` com filtro de tenant:

```
public interface WidgetRepository extends JpaRepository<Widget, UUID> {
    Page<Widget> findByOrganization_IdAndDeletedAtIsNull(UUID orgId, Pageable p);
    Optional<Widget> findByIdAndOrganization_IdAndDeletedAtIsNull(UUID id, UUID orgId);
}
```

4. **DTOs** em `dto/`: `WidgetRequest` (Jakarta Validation) e `WidgetResponse` (records).

5. **Service** em `service/` com `@Transactional`, recebendo `organizationId` como primeiro parâmetro; `delete` preenche `deletedAt`.
6. **Controller** em `controller/` fino, lendo `TenantContext`, anotado Swagger:

```
@RestController @RequestMapping("/api/v1/widgets")
@RequiredArgsConstructor @Tag(name = "Widgets", description = "Endpoints for managing
widgets")
public class WidgetController { ... }
```

Frontend:

7. **Service** estendendo `BaseService<Widget>` fixando path `widgets`.
8. **Componente de lista** standalone usando `ListPageComponent` (`columns`, `kpis`, outputs → service + `FormDialogComponent`).
9. **Rotas** `features/widgets/widgets.routes.ts` exportando `WIDGETS_ROUTES`.
10. **Registro** — rota lazy em `app.routes.ts` e item de navegação em `shell.component.ts` na seção correta (CRM/Operações/Financeiro/Admin).

9.2 Convenções obrigatórias

- **Toda consulta filtra por organização.** Repositório sem filtro vaza tenants. Sem rede de proteção no banco.
- **Toda exclusão é lógica.** Preencha `deletedAt`; filtre `DeletedAtIsNull`.
- **Entidade nunca sai pela API.** Controllers respondem DTOs; serializar entidade com lazy quebra ou expõe demais.
- **Tenant vem do token.** Nunca aceite `organizationId` do body/query.
- **Regra de negócio no service.** Controllers só orquestram; repositórios só consultam.
- **Erros de negócio em português.** `BadRequestException` (regra violada) e `ResourceNotFoundException` (não encontrado); `GlobalExceptionHandler` cuida do status.

9.3 Testes

Backend:

```
cd backend && mvn test
```

Stack JUnit 5 (`spring-boot-starter-test`), Mockito, `spring-security-test`, H2 em memória. **144 testes em 23 classes:** controllers (auth, users, clients, deals, projects, proposals, dashboard, financeiro) e services (`ProposalService`, `PipelineEngine`, `AnalyticsService`, `DealService`, `LeadService`, `ProjectService`, `ClientService`, `LegalProcessService`, etc.) apoiados em `config/BaseServiceTest.java`.

Sem cobertura: `LeadScoringService`, `CommissionService`, `LgpdService`, `CampaignService` e CRUDs simples. Build Docker roda com `-DskipTests`; testes no CI/local.

Frontend:

```
cd frontend && npm test # Karma + Jasmine - sem specs relevantes
```

9.4 Build

```
cd backend && mvn package # JAR em target/  
cd frontend && npm run build -- --configuration production # bundle em dist/crm-axe-frontend/browser
```

9.5 Commits

Formato convencional `<tipo>: <descrição>` com `feat`, `fix`, `refactor`, `docs`, `test`, `chore`, `perf`, `ci`. Descrição no imperativo.

9.6 Onde procurar cada coisa

Preciso de...	Vá para
Contrato de endpoint	Swagger UI ou <code>controller/</code> + <code>dto/</code>
Regra de negócio	<code>service/</code>
Estrutura de tabela	<code>resources/db/migration/</code>
Isolamento tenant	<code>auth/security/TenantFilter</code> , <code>TenantContext</code> , <code>commons/entity/BaseEntity</code>
Autenticação	<code>auth/security/</code> , <code>auth/service/AuthService</code>
Tela de listagem	<code>shared/list-page/</code> + componente do módulo
Navegação	<code>app.routes.ts</code> , <code>shell/shell.component.ts</code>
Tokens visuais	<code>DESIGN.md</code>

10. Pontos de Atenção

Itens verificados no código em 07/08/2026, com arquivo onde ocorrem. Ordem por severidade.

Resolvido desde a primeira versão: duplicidade `V26__create_google_tokens.sql` vs `V26__proposals_link_and_deal_stage_history.sql` (Flyway não subia). Primeiro passou a `V27`. Ao provisionar ambiente com versão antiga aplicada, conferir `flyway_schema_history`.

Crítico

API não aplica autorização por papel — nenhum controller usa `@PreAuthorize` / `@Secured` nem regra por papel em `SecurityConfig` (só `anyRequest().authenticated()`). Papel entra no token como authority mas nada consome. `adminGuard` do frontend esconde `/users` e `/integrations`, mas qualquer

autenticado pode chamar `POST /api/v1/users` direto. **Correção:**

`@PreAuthorize("hasAnyAuthority('ADMIN','SUPER_ADMIN')")` nos controllers administrativos + `@EnableMethodSecurity`.

Segredo JWT de desenvolvimento versionado — `application.yml` traz `jwt.secret` embutido. Perfil `render` exige `JWT_SECRET` via env, mas deploy sem perfil `render` usaria segredo público permitindo forjar tokens. **Correção:** remover default e falhar na inicialização sem `JWT_SECRET`.

Alto

`JWT_EXPIRATION` **sem efeito** — `JwtUtil` injeta `@Value("${jwt.expiration-ms:86400000}")`, mas YAMLS definem `jwt.expiration`. Propriedade lida não existe, sempre usa default 24 h, inclusive no Render onde `JWT_EXPIRATION` está no `render.yaml`. **Correção:** alinhar nomes.

CORS aberto com credenciais — `CorsConfig` com `allowedOriginPatterns("*")` + `allowCredentials(true)` + `allowedHeaders("*")`. Qualquer origem chama API. Autenticação por header `Authorization` (não cookie) reduz risco imediato, mas não deve ir a produção. **Correção:** restringir origens por perfil.

Integrações Google são mocks no controller — `IntegrationController` responde `/google/status`, `/google/connect`, `/google/disconnect`, `/google/calendar` com dados fixos (e-mail `contato@axelpro.com.br`, eventos estáticos); `/ai/generate` devolve texto pré-fabricado. Existe `GoogleIntegrationService` funcional (OAuth2 completo, refresh, Calendar/Contacts/Gmail, persistência `google_tokens`) mas **controller não o utiliza**. Callback `/api/v1/integrations/google/callback` referenciado como redirect URI não está implementado. **Correção:** ligar controller ao service e configurar `google.client-*`.

Envio de campanha carrega todos os clientes — `CampaignService.sendCampaign` usa `clientRepository.findAll()` e filtra org em memória, enquanto leads usa consulta filtrada. Não vaza dados na resposta, mas traz clientes de todos os tenants à JVM (custo cresce com base total). Além disso inclui **todos** leads/clientes com e-mail/telefone sem segmentação e simula envio (grava `campaign_recipients`, marca como enviada, nenhuma mensagem sai). **Correção:** `findByOrganization_IdAndDeletedAtIsNull` e explicitar na UI que envio é simulado.

Médio

Logs DEBUG na config padrão — `application.yml` com `org.springframework.security` e `org.springframework.web` em `DEBUG` (útil em dev, ruidoso/revelador em outros). Perfil `render` corrige para `INFO`.

`open-in-view: true` — mantém sessão Hibernate aberta na renderização da resposta; mascara `LazyInitializationException` e favorece N+1 na serialização.

`flyway.repair()` **a cada inicialização** — `FlywayMigrationConfig` executa `repair()` antes de `migrate()` sempre; reescreve checksums divergentes em vez de falhar — edições em migrations aplicadas passam despercebidas, ambientes divergem silenciosamente.

Status como string livre — `Project.status`, `Invoice.status`, `Contract.status` são `String` sem validação, enquanto `LeadStage`, `ProposalStatus`, `TaskStatus` usam enums. Valores gravados como

vierem do cliente.

Cobertura de testes parcial — 144 testes cobrem controllers principais e services de maior risco (`ProposalService`, `PipelineEngine`, `AnalyticsService`, `DealService`, `ProjectService`); sem cobertura `LeadScoringService`, `CommissionService`, `LgpdService`, `CampaignService`. Docker com `-DskipTests`.

Exceções fora do padrão — `ProspectService.promoteToLead` e `CampaignService.sendCampaign` lançam `IllegalStateException` (não tratada pelo `GlobalExceptionHandler` → 500 em vez de 400 legível). Demais regras usam `BadRequestException`.

Baixo

Marca inconsistente — produto **Axel CRM** (repo, `PRODUCT.md`), shell exibe "IBCAPPA CRM" (placeholder legado).

Header `X-Tenant-Id` sem uso — `tenantInterceptor` envia `X-Tenant-Id` e `CorsConfig` expõe, mas backend nunca lê (tenant vem do JWT). Inofensivo, mas sugere mecanismo inexistente.

Token de proposta pública sem expiração — `publicToken` UUID permanente; quem tem o link acessa proposta/PDF indefinidamente, mesmo expirada/rejeitada. Considerar validade/revogação.

README desatualizado — marcava "Calendar & tasks" e "File attachments" como pendentes, mas existem (`/calendar-events`, `/tasks`, `/clients/{id}/attachments`).

11. Onboarding Rápido

Resumo operacional para quem acabou de clonar o repo. Detalhes nos caps. 9 e 8.

Stack: Java 21 + Spring Boot 4.0.7, Angular 18.2 Material + Chart.js, PG 16 + Flyway V1–V29, JWT Auth0 4.4.0, SpringDoc 2.8.0.

Arquitetura em 1 diagrama:

```
Angular SPA (:4200) —HTTP /api/v1→ Spring Boot (:8080)
      |
      | JwtAuthenticationFilter
      |
      | TenantFilter → TenantContext
      |
      | Controller → Service (@Transactional) → Repository
      |
      | (findByOrganization_IdAndDeletedAtIsNull) → PostgreSQL
```

Pontos de entrada chave:

- API: `backend/src/main/java/com/axelcrm/controller/` (43 controllers `/api/v1/...`)
- Regras: `backend/.../service/` (44 services)

- Modelo: `backend/src/main/resources/db/migration/` (fonte da verdade)
- Segurança: `backend/.../auth/security/` (`SecurityConfig` , `JwtAuthenticationFilter` , `TenantFilter` , `TenantContext`)
- Rotas SPA: `frontend/src/app/app.routes.ts` + `features/<modulo>/<modulo>.routes.ts`
- Shell: `frontend/src/app/shell/shell.component.ts`
- Config: `backend/src/main/resources/application*.yaml` , `frontend/src/environments/`

Mapa de diretórios backend:

```

backend/src/main/java/com/axelcrm/
├─ controller/ → endpoints REST finos
├─ service/ → regras @Transactional
├─ repository/ → Spring Data JPA com filtro tenant
├─ entity/ → JPA (estendem BaseEntity)
├─ dto/ → records XxxRequest/XxxResponse
├─ auth/ → auth vertical completa
├─ config/ → CORS, OpenAPI, JPA auditing, Flyway
├─ commons/ → BaseEntity, exceções, handler
└─ analytics/ → views de relatórios

```

Ciclo de vida de request: `GET /api/v1/clients` → `SecurityConfig` exige auth → `JwtAuthenticationFilter` valida JWT → `TenantFilter` resolve org → `ClientController.findAll(Pageable)` lê `TenantContext` → `ClientService.findAll(orgId, page)` → `findByOrganization_IdAndDeletedAtIsNull` → DTO `ClientResponse` .

Convenções: `@RequiredArgsConstructor` + `@Data` (Lombok), DTOs records, `@Tag` / `@Operation` , erros pt-BR, soft delete sempre.

Tarefas comuns:

Tarefa	Comando
Rodar backend	<code>cd backend && mvn spring-boot:run</code> (DB <code>localhost:5440/axelcrm</code>)
Rodar frontend	<code>cd frontend && npm install && ng serve</code>
Testes backend	<code>cd backend && mvn test</code> (144/23)
Build	<code>mvn package</code> / <code>npm run build -- --configuration production</code>
Swagger	<code>http://localhost:8080/swagger-ui.html</code>
Nova migration	<code>V<n>__descricao.sql</code> em <code>db/migration/</code> (nunca editar aplicada)

12. Anexos

12.1 ADRs (Architecture Decision Records)

Local: `docs/adr/` no formato MADR:

ADR	Decisão
001	Multi-tenancy por coluna <code>organization_id</code> (schema compartilhado) — simplicidade e custo, sem RLS
002	Autenticação JWT stateless (HMAC256, sem sessão) — SPA stateless
003	Exclusão sempre lógica (<code>deletedAt</code>) — auditabilidade e recuperação
004	Camada de DTOs (records) — nunca expor entidade JPA
005	Flyway com <code>ddl-auto: none</code> — controle total do schema
006	Frontend Angular standalone + lazy loading — chunks por módulo
007	Deploy Render Blueprint + Docker — infra como código gratuita
008	Erros de negócio em português — mensagens chegam ao operador
009	Aceitação de proposta na página pública — vinho sem login, sem expiração

Ver `docs/adr/README.md`.

12.2 WhatsApp Inbox

Novo em V29 e `features/whatsapp/`:

- Conexão por QR code (`whatsapp-qr-dialog`), status de vínculo (`whatsapp-link-dialog`), inbox com conversas.
- Service `frontend/src/app/features/whatsapp/whatsapp.service.ts`, rotas `whatsapp.routes.ts`, componente `whatsapp-inbox.component.ts`.
- Specs: `docs/specs/whatsapp-inbox.md`, `whatsapp-inbox-frontend-report.md`, `whatsapp-atendente.md`; plano `docs/plans/whatsapp-inbox-impl.md`.

12.3 Geração de PDF

- `docs/pdf/` com `gerar-pdf.ps1`, `build-pdf.js`, `cover.html`, `cover-manual.html`, `pdf.css`.
- Geração requer Node.js, Pandoc e Chrome headless. Script concatena capítulos, converte links em âncoras internas:

```
powershell -File docs\pdf\gerar-pdf.ps1 # ambos
powershell -File docs\pdf\gerar-pdf.ps1 -Only manual # só manual
```

Saída: `docs/axel-crm-documentacao.pdf` (42 p) e `docs/axel-crm-manual-do-usuario.pdf` (31 p) com capa e sumário navegável.

12.4 Documentos relacionados

Documento	Conteúdo
<code>README.md</code> (raiz)	Execução rápida e deploy
<code>PRODUCT.md</code>	Definição de produto, posicionamento e princípios
<code>DESIGN.md</code>	Design system (tokens, tipografia, componentes)
<code>docs/README.md</code>	Índice dos 10 capítulos técnicos
<code>docs/manual-do-usuario/README.md</code>	Manual do usuário (9 capítulos)
<code>docs/onboarding.md</code>	Onboarding resumido

12.5 Como manter esta documentação

Esta documentação descreve comportamento **verificado no código**. Ao alterar contratos de API, modelo de dados ou regras de negócio, atualize o documento correspondente e esta consolidação no mesmo commit. Se algo estiver incompleto ou mockado, registre em **Pontos de Atenção** em vez de descrever intenção como pronta. Links entre seções usam âncoras internas — mantenha os títulos.

Sugestão de formatação: após editar, execute `npm run format` (Prettier/ESLint) se configurado, ou `powershell -File docs\pdf\gerar-pdf.ps1` para regenerar os PDFs.

*Fim da documentação completa. Para dúvidas técnicas, comece pelo **Onboarding Rápido** e depois navegue aos capítulos específicos acima. Para uso operacional, consulte o **Manual do Usuário**.*